Analisi ad oggetti: Metodo CRC

- Modellazione CRC (Class-Responsibility-Collaboration):
  - Metodo per identificare e organizzare classi e requisiti di un sistema o prodotto.
  - Un modello CRC consiste in una raccolta di schede che rappresentano le classi.
- Scheda CRC:
  - Parte superiore:
    - Nome della classe, superclassi e sottoclassi.
      - Sezione Class:
        - Individuazione e definizione delle classi su schede di cartoncino.
        - Definizione delle gerarchie di ereditarietà.
  - Parte centrale:
    - Responsabilità (a sinistra).
      - Sezione Responsibility:
        - Specifica delle responsabilità di ogni classe.
        - Porta alla definizione dell'interfaccia della classe.
    - Collaboratori necessari per realizzare le responsabilità (a destra).
      - Sezione Responsibility:
        - Specifica delle responsabilità di ogni classe.
        - Porta alla definizione dell'interfaccia della classe.
- Scopo delle schede CRC:
  - Presentazione organizzata delle classi.
  - Determinazione delle responsabilità e collaboratori delle classi.
- Vantaggi delle schede CRC:
  - Fallire anticipatamente, spesso e senza costo.
  - Evitare riorganizzazione massiva del codice sorgente.
- Fasi del CRC
- Caratteristiche del Metodo CRC:
  - Indipendente dal linguaggio di programmazione.
  - Prevede le seguenti Attività del metodo:
    - Identificazione delle classi dai requisiti del sistema.
    - Determinazione delle responsabilità di ogni classe.
    - Determinazione delle collaborazioni tra classi.
    - Identificazione delle gerarchie di classi.
    - Identificazione dei sottosistemi.
    - Specifica dettagliata dei protocolli (interfacce) per le classi.
  - Notazione grafica utilizzata nel metodo CRC:
    - Schede per le classi.
    - Grafici di gerarchia (notazione UML).
    - Grafici di collaborazione.

Presented with xmind

Fasi del CRC

Individuazione delle classi

- Fonti per l'individuazione delle classi
  - **Documento dei requisiti**
  - **Casi d'uso o user stories**
  - **Colloqui con i futuri utenti**
  - **Altri sistemi nello stesso dominio**
  - **Documenti tecnici che descrivono il dominio**
  - **Diagrammi di vario tipo per descrivere il sistema**
- Processo di individuazione delle classi
  - Elencare i nomi presenti nei documenti e convertirli al singolare
  - Eliminare i nomi non riferibili a classi e attributi
  - Scegliere un solo termine significativo per concetti simili
  - Prestare attenzione agli aggettivi e agli attributi per identificare nuove classi
  - Rendere attive ed esplicite le frasi passive o con soggetti fuori dal sistema
  - Studio e costruzione del modello per soffrisce l'OOD e scoprire nuove classi
  - Individuazione delle gerarchie di ereditarietà per scoprire nuove classi
- Tipologie delle classi
  - **Classi di attori**
    - Rappresentano persone o organizzazioni che hanno un ruolo nel sistema.
    - Esempi di classi di attori includono persone, organizzazione (agenzia, associazione, ente, fondazione, società).
  - **Ruoli degli attori**
    - Arricchiscono come gli attori partecipano al sistema e definire il loro ruolo.
    - Alcuni esempi di ruoli degli attori includono cliente, amministratore, cassiere, cliente, compratore, ecc.
  - **Classi di posti**
    - Rappresentano le ubicazioni ove risiedono le cose o che contengono altri oggetti.
    - Alcuni esempi di classi di posti sono aeroporto, agenzia, banca, ecc.
  - **Eventi/transazioni**
    - Rappresentano avanti o transazioni che il sistema deve memorizzare.
    - Possono derivare da diversi origini come:
      - Dati inventi dall'utente attraverso un' interfaccia grafica (GUI).
      - Altri oggetti che controllano e registrano l' avvenimento di un evento significativo.
      - Altri sistemi.
    - Esempi di eventi/transazioni includono acquisti, affitti, ordini, pagamenti, prenotazioni, richieste, vendite.
    - Spesso le transazioni coinvolgono di uno o più elementi tipo transazione.
  - **Modelli e loro elementi specifici**
    - Rappresentano oggetti descrittivi e le loro specifiche tecniche.
    - Ad esempio, un modello di auto come la Fiat 500X e le auto specifiche di quel modello che il sistema deve registrare.
    - Esempi di attributi possono includerle la categoria presso elementi, la categoria tasso-tassa specifica, la descrizione del compito compito specifico e il modello aereo-aereo specifico.
  - **Sistemi e dispositivi**
    - Rappresentano altri sistemi e dispositivi con cui il sistema deve interagire.
    - Possono includerle attuatori, DBMS, Internet, LAN, reti, sensoriali, sensori e altri.
  - **Cose tangibili**
    - Rappresentano oggetti concreti del dominio del problema che il sistema deve gestire.
    - Esempi di classi di cose tangibili sono mobili, auto, treni, PC, telefoni cellulari.
  - **Oggetti contenitori**
    - Rappresentano oggetti che contengono fisicamente o virtualmente altri oggetti.
    - Spesso queste classi possono anche essere considerate come posti.
    - Alcuni esempi includono aerei, aeroporti, aerosol, banche, cameriere, depositi, edifici, garage, hangar, magazzini, negozi, ospedali, corpi docenti, dipartimenti, scuole, scaraci, squadre.
- Tipologie degli Attributi
  - **Attributi a valore singolo**
    - Un singolo valore associato a un'istanza della classe.
  - **Gruppi di valori strettamente legati**
    - Gruppo di valori strettamente correlati tra loro.
    - Esempio: indirizzo.

Tipologia	Attributi tipici
Attori	codice fiscale, indirizzo, matricola, nome, telefono,
Ruoli degli attori	ruolo, livello autorizzazione, numero, ora, parole d'ordine
Posti	altezza, indirizzo, latitudine, longitudine, numero, nome
Cose tangibili	codice, colore, data, marca, modello, numero, ora,
Eventi e transazioni	codice, data, importo, nome, ora, stato,
Modelli	codice universale prodotto (UPC), codice, descrizione, dimensioni, nome, peso,
Elementi specifici	data acquisto, matricola, numero serie, targa
Sistemi e Dispositivi	nome, numero, stato operativo

Attributi tipici in base alla tipologia della classe

Criteri di scelta per inclusione di una classe

- **Competenza del sistema o interazione**
  - L'oggetto è di competenza del sistema?
  - L'oggetto deve interagire con il sistema?
- **Responsabilità dell'oggetto**
  - Il sistema deve ricordare qualcosa associato all'oggetto?
  - Validità e importanza della classe senza memoria associata.
  - Presenza di un oggetto duplicato con la stessa funzione.
- **Numero di istanze**
  - Classi con un solo oggetto sono sospette.
  - Eccezione per classi di alto livello come Imprese, Aeroporti, Università.
  - Favorire classi con più istanze.
- **Applicabilità delle responsabilità**
  - Responsabilità sempre applicabili o solo per alcuni oggetti della classe.
  - Considerare l'ereditarietà per distinguere le responsabilità.
- Nomi composti che rappresentano un singolo concetto.

Compilazione scheda delle classi

Classe: **Evento** Tipo/classe: **Nessuna** Astratta:

Descrizione:

Un evento rappresenta qualche cosa di significativo per il sistema che accade ad un preciso istante di tempo e che viene in cale ad interagire.

Responsabilità

1. Un Evento conosce la propria data e ora.
2. Un Evento conosce il proprio tipo e la descrizione che, in conseguenza di un preciso accadimento, notano le reti del sistema.

Scheda delle classi

- **Dopo avere identificazione delle classi**
- Compilazione delle schede
  - **Nome della classe sulla scheda**
  - **Descrizione dello scopo della classe sul retro della scheda**
- Scopo della scheda delle classi
  - Nucleo della documentazione dettagliata della classe
  - Fornisce informazioni essenziali sulla classe

Identificazione responsabilità

- Classi del sistema con due tipi di responsabilità
  - **Mantenere informazioni (stato).**
  - **Effettuare azioni (comportamento).**
- **Identificazione delle responsabilità**
  - Esaminare i requisiti per elencare i verbi delle azioni.
  - Associare le informazioni gestite dagli oggetti del sistema.
  - Simulazione mentale o diagrammi per individuare le azioni del sistema.
  - Classi già identificate devono avere almeno una responsabilità.
  - Determinare le informazioni e azioni necessarie per ogni classe.
- **Assegnazione delle responsabilità**
  - Distribuire l'intelligenza in modo bilanciato nel sistema.
  - Assegnare responsabilità al livello più generale possibile.
  - Collegare comportamento e informazioni necessarie.
  - Mantenere le informazioni su una cosa in un solo posto.
- Principi di buona progettazione
  - I principi di buona progettazione possono essere applicati per decidere come assegnare le responsabilità del sistema.
  - Non esiste un modo univocale per determinare le classi e le loro responsabilità, ma diverse soluzioni orientate agli oggetti possono essere valide.
  - È importante notare che ci sono molte più soluzioni inefficienti o scarsette rispetto a quelle corrette.
  - I principi di buona progettazione aiutano a raggiungere un design pulito, mantenibile e flessibile.
- Ulteriori responsabilità
  - **È possibile identificare ulteriori responsabilità esaminando le relazioni tra le classi.**
    - La relazione "è un tipo di" indica l'esistenza di una relazione superclasse-sottoclasse, dove le due classi condividono spesso delle responsabilità.
    - La relazione "è analoga a" indica l'esistenza di una superclasse comune, in cui vanno assegnate delle responsabilità.
    - La relazione "è parte di" indica che un'istanza di una classe deve fare in modo che le responsabilità di contenerti e controllarti, senza necessariamente una relazione di ereditarietà.
  - **La responsabilità della classi devono essere segnate in modo conciso nelle schede delle classi, una per riga.**
    - È importante evitare che una classe abbia troppe responsabilità, poiché potrebbe indicare un eccessivo livello di dettaglio nel progetto o una concentrazione eccessiva di "intelligenza" nella classe stessa.
    - In caso di troppe responsabilità, conviene suddividere la classe in più classi che cooperano tra loro, distribuendo le responsabilità in modo bilanciato.

Identificazione collaborazioni

- **Collaborazioni**
  - Le collaborazioni sono interazioni tra classi
  - Una classe può collaborare con altre per soddisfare le sue responsabilità
  - Le classi soddisfano le responsabilità tramite azioni interne o servizi di altre classi
  - Ogni collaborazione coinvolge un client e un server
  - La collaborazione è un contratto tra le classi coinvolte
  - Una classe può richiedere nessuna, una o più collaborazioni per soddisfare una responsabilità.
  - Le collaborazioni mostrano il flusso di controllo e lo scambio di informazioni tra gli oggetti
- Identificazione delle collaborazioni
  - Domande per ogni responsabilità della classe
    - Può soddisfare autonomamente la responsabilità?
    - Cosa serve alla classe?
    - Quali altre classi possono fornire il supporto necessario?
  - Considerare altre classi coinvolte
    - Funzioni e conoscenze della classe in questione
    - Dipendenza da altre classi per risultati o informazioni
    - Classe non interagisce con altre classi, potrebbe non essere necessaria
- **Associazione delle collaborazioni alle responsabilità**
  - Utilizzare una scheda delle classi
  - Inserire il nome del server nel campo destro della scheda client, accanto alla responsabilità nella scheda del client
  - Indicare la dipendenza della classe client dalla classe server
  - Elencare i nomi delle classi server coinvolte se necessario
  - Specificare se una collaborazione è richiesta da molte responsabilità
  - Segnalare collaborazione tra istanze della stessa classe (client o server) se necessario

Affinamento delle Gerarchie

- **Caratteristiche**
  - Effettuata dopo che l'analisi preliminare del sistema è completata
  - Successivamente ci si concentra sulla progettazione dettagliata
- **Verifica e modifica delle gerarchie di classi**
  - Massimizzare (benefici) delle tecniche ereditate agli oggetti
  - Completamento dei diagrammi delle classi aggiungendo specifiche importanti
    - Ad esempio specificare se una classe è astratta o concreta
  - Creazione di superclasse comune (astratta) per gruppi di classi con stessa responsabilità
  - Creazione di molte superclasse se rappresentano la semantica reale del sistema
- **Fattorizzazione delle responsabilità comuni**
- **Classe astratte e classi concrete**
  - Classi astratte non devono ereditare da classi concrete
  - Classi astratte gestiscono le responsabilità indipendentemente dall'implementazione
- **Eliminazione delle classi non funzionali**
  - Eliminazione di classi senza responsabilità aggiuntive
  - Attribuzione alle responsabilità ereditate come funzionalità aggiuntive
- **Dalle schede CRC ai diagrammi UML**
  - **Metodo originale delle schede CRC (1990)**
    - Identificazione delle relazioni di ereditarietà
    - Identificazione dei sottosistemi
    - Specifica dei protocolli per ogni classe
    - Codifica delle classi
    - Identificazione dei pacchetti come sottosistemi
    - Specificazione dei diagrammi delle classi UML del sistema
- **Diagramma delle classi UML**
  - Ogni classe identificata diventa una classe UML nel diagramma
  - Rappresentazione delle gerarchie di ereditarietà con relazioni di ereditarietà in UML
  - Attributi diventano dati elementari nella classe UML
  - Responsabilità complesse diventano operazioni nella classe UML
  - Collaborazioni diventano relazioni (associazione, aggregazione, composizione) nel diagramma delle classi UML
- **Pacchetti (Package)**
  - Suddivisione dei modelli in pacchetti o sottosistemi coesiganti
  - Facilitazione della comprensione e delle attività successive
  - Utile durante l'analisi orientata agli oggetti e in fase di prototipazione

Identificazione Interfacce delle classi

- Una volta identificate e definite le responsabilità delle classi, queste possono essere convertite in un interfaccia della classe
- **Contenuto dell'interfaccia di una classe**
  - **Metodi pubblici**
    - Nome del metodo
    - Nome e tipi dei parametri
    - Tipo di ritorno
    - Descrizione del metodo
- **Linee guida per l'identificazione delle interfacce**
  - Raggruppamento delle responsabilità utilizzate dagli clienti
  - Massimizzazione della coesione delle classi
  - Minimizzazione del numero di metodi
  - Raggruppamento delle responsabilità private
- Le operazioni sono divise in due categorie
  - **Operazioni algoritmicamente semplici**
    - **Operazioni comuni a tutti gli oggetti**
    - Non compaiono nel diagramma delle classi
    - Esempi di operazioni: creazione, cancellazione, lettura, scrittura, aggiunta, eliminazione, dimensione, connessione, disconnessione
    - **Specifiche di ciascun oggetto**
    - Richiedono calcoli, selezioni o controlli su valori esterni
    - Identificazione delle operazioni considerando richieste, calcoli, selezioni e controlli
    - Esempi di verbi che aiutano nell' identificazione: attiva, calcola, controlla, determina, disattiva, misura, qualifica, riporta, seleziona, trova
  - **Operazioni algoritmicamente complesse**

Assegnazione delle operazioni

- **Principio generale**
  - Distribuire in modo bilanciato le operazioni nel sistema
  - Considerare i dati necessari a ciascuna operazione
  - Mantenere la lunghezza media delle operazioni entro le 4-8 istruzioni
- **Applicazione dell'ereditarietà nel posizionamento delle operazioni**
  - Posizionare le operazioni più generali nella parte superiore
  - Posizionare le operazioni specializzate nella parte inferiore
- **Ottimizzazione delle responsabilità degli oggetti**
  - Valutare se un oggetto può svolgere altre operazioni sui propri dati
  - Seguire la legge di Demeter per distribuire meglio le responsabilità
- **Organizzazione delle collezioni**
  - Le collezioni devono occuparsi solo delle operazioni relative all'insieme
  - Inserire il lavoro specifico di ogni componente all'interno della collezione
- **Nomenclatura delle operazioni**
  - Utilizzare il vocabolario standard del dominio del problema per i nomi delle operazioni
  - Utilizzare verbi all'imperativo o in terza persona (coerentemente nel sistema
- **Descrizione dell'operazione**
  - Fornire una breve descrizione dell'operazione
  - Specificare i parametri di input/output e il loro tipo e significato
  - Definire le pre-condizioni e le post-condizioni dell'operazione
  - Indicare gli eventi che attivano l'operazione e le sue applicabilità negli stati
- **Visibilità dell'operazione**
  - Definire la visibilità dell'operazione come pubblica, protetta o privata
- **Documentazione dell'operazione**
  - Documentare l'operazione con pseudo-codice, diagrammi o bozzetti a scelta di prototipazione

Definizione del OOD: il Componente del Dominio del Problema

- **Ristrutturazione del modello OOA**
  - Passaggio a un modello OOD (Problem Domain Component)
  - Considerazione del linguaggio di implementazione e delle prestazioni
  - Specifica dell'implementazione delle relazioni di associazione e dipendenza

Definizione del OOD: il Componente di Interazione Umana

- **Componente di interazione Umana (HIC)**
  - Progetto dell'interazione con l'utente
  - Organizzazione delle classi applicative che eseguono le istruzioni dell'utente e compiono i report
  - Interazione con le classi del PDC (Problem Domain Component)
- **Generazione dell'interfaccia utente (GUI)**
  - Creazione visuale delle finestre del sistema
  - Associazione di messaggi agli eventi che possono verificarsi nelle finestre
  - Invio dei messaggi agli oggetti del sistema
  - Oggetti rappresentati dalle classi dell'HIC

Presented with xmind